ZENITH SCHOOL OF AI / OPEN-SOURCE EVALUATION

FINTECH LOYALTY

RewardBook

Rewards Rules & Repeat-Transaction Alerts

FORMAT	DURATION	LANGUAGE	DATA
Individual	24-hour individual evaluation	Python 3.10+	Synthetic / offline

Official evaluation repository: https://github.com/sunitazenith07-lang/RewardBook-Open-Source-Evaluation

Open the repository in a browser, then read `STUDENT_PROBLEM_STATEMENT.md` and the README before changing code.

1. Scenario

A loyalty product awards points for eligible purchases. Promotions may add a bonus or cap. Purchases can later be refunded; duplicate transaction events must not award points twice. Operations also need transparent warnings for possible same-account, same-day, same-amount repeats.

This is a fictional offline service inspired by common rewards products. It is not any company's internal code and uses no real financial or customer data.

2. Required behaviour

1. Calculate points using integer arithmetic: `base_points = floor(amount_paise / 10000) * points_per_100_rupees`; `bonus_points = floor(base_points * (100 + bonus_percent) / 100)`; apply optional `cap_points` last.
2. Invalid or negative amounts/rules must not generate points. A cap of zero means zero points.
3. Only captured purchases earn points. Declined, pending and unknown events must not.
4. Process each `event_id` once so replaying purchase/refund events cannot alter balances twice.
5. Reverse points proportionally for refunds using integer arithmetic, never reverse more than awarded and never allow a negative balance.
6. Write an auditable ledger entry for each accepted purchase/refund event.
7. Flag possible repeats by account, date and amount using the supplied threshold. Alerts are for human review only; never block a transaction or automatically accuse a customer.
8. Handle empty input, missing identifiers, unknown transactions and boundary values safely.

3. Setup and first checks

Use Python 3.10 or newer and Git. From the repository root, run the following commands in order:

```
python -m pip install -r requirements-dev.txt
python -m pytest -q
python -m main
```

4. Work to submit

- Read the source and run the starter tests before editing.
- Use the requirements and tests as the acceptance contract. Trace failures to their root causes, fix the underlying behaviour, and avoid hard-coding sample output.
- Add regression tests and document at least five edge cases in `TESTING.md`.
- Submit one pull request to the original repository with the fixes, tests and a brief explanation. Do not merge it.
- Include `REWARD_RULE_NOTES.md` with one worked points calculation, one partial-refund example and three edge cases.

5. Optional stretch work

Choose at most one stretch item, and only after the required behaviour is correct. Stretch work is not required for a strong score.

- Generate a CSV account statement.
- Make the repeat threshold configurable.
- Add a rewards summary by category.

6. Testing and GitHub Actions

Some visible tests are expected to fail on the untouched starter because key behaviours are incomplete. The exact initial test count is not fixed. GitHub Actions runs on pushes and pull-request updates and reports visible test counts. A red check before your fixes is expected; make the visible tests pass without deleting, skipping, weakening or bypassing them. Do not alter the workflow to evade failures.

7. Individual-work policy

AI chatbots, AI coding assistants, generated patches/tests and asking another person to write or debug your solution are prohibited. Ordinary non-AI language/framework documentation is allowed. Do not upload challenge code or data to external services. You must be able to explain your changes and make a small live modification if asked. A code walkthrough can support integrity checks, but no automated detector can prove whether AI was or was not used.

8. Evaluation rubric - 100 marks

Criterion	Marks
Understanding code and data flow	**15**
Correct points calculation and financial edge cases	**25**
Idempotency, refunds and ledger integrity	**25**
Testing and root-cause analysis	**15**
Code quality	**10**
Pull request, notes and live explanation	**10**
Total	**100**

Technical expectation: Marks reward correct reasoning and behaviour that matches this written contract, not line count. A green test suite alone does not guarantee full marks; be ready to explain your choices, test strategy and known limitations.

Repeat alerts are only signals for human review. The system must not automatically block or accuse a customer.

This is a simulated startup-style challenge inspired by common product/operations problems. It is not a confidential or official problem statement from any named company. All datasets are artificial.